

Empirical Big-O Project Report

Jason Zhang, Kyle Yang, and Aidan Roberts
TJHSST

March 2025

Abstract

We aim to provide an empirical analysis on time complexity (Big-O) values of common sorting algorithms implemented in Java.

1 Introduction

The purpose of this study is to use an empirical approach for a comparison with the theoretical time complexity analysis for common sorting algorithms in AP Computer Science A+ and Data Structures. This is not meant to serve as a formal paper, but instead as a technical report on our findings.¹

2 Empirical Results

2.1 Methodology

Our group was provided with a Jar file [Tea] containing sorts and system time. To begin, we followed the given procedures. We tested the sorting algorithms on random doubles with array size $n \in \{500, 1000, 2000, 4000, 8000\}$

We then decompiled the Jars by extracting the class files and using a decompiled. Upon this, we made slight modifications for ease of collecting data (such as a button that would allow us to run the same sorting algorithm 5000 times).

The algorithms used were implemented in Java following the psuedocode outline.

¹Due to time constraints there may be typographical errors within this report. The authors would like to apologize if this occurs and they hope it does not detract from the content.

Data: An array of non-negative numbers (0-indexed). n is the length of the array.

Result: A sorted array of numbers.

```
for  $i \leftarrow 0$  to  $n - 1$  do
    maxPos  $\leftarrow 0$ ;
    for  $j \leftarrow 1$  to  $n - i - 1$  do
        if  $array[j] > array[i]$  then
            maxPos  $\leftarrow j$ ;
        end
    end
    swap(maxPos,  $n - i - 1$ );
end
```

Algorithm 1: Selection Sort

Data: An array of non-negative numbers (0-indexed). n is the length of the array.

Result: A sorted array of numbers.

```
for  $i \leftarrow 1$  to  $n - 1$  do
    temp  $\leftarrow array[i]$ ;
    index  $\leftarrow i - 1$ ;
    while  $index \geq 0$  and  $array[index] > temp$  do
        array[index + 1]  $\leftarrow array[index]$ ;
        index  $\leftarrow index - 1$ ;
    end
    pos  $\leftarrow index$ ;
    array[pos]  $\leftarrow temp$ ;
end
```

Algorithm 2: Insertion Sort

Data: An array of non-negative numbers (0-indexed). n is the length of the array.

Result: A sorted array of numbers.

copyBuffer $\leftarrow$ empty array of length n ;

Function Merge($low, middle, high$)

```
     $i_1 \leftarrow low$ ;  
     $i_2 \leftarrow middle + 1$  ;  
    for  $index \leftarrow low$  to  $high$  do  
        if  $i_1 > middle$  then  
            copyBuffer[index]  $\leftarrow$  array[ $i_2$ ];  
             $i_2 \leftarrow i_2 + 1$ ;  
        else if  $i_2 > high$  then  
            copyBuffer[index]  $\leftarrow$  array[ $i_1$ ];  
             $i_1 \leftarrow i_1 + 1$ ;  
        else if array[ $i_1$ ] < array[ $i_2$ ] then  
            copyBuffer[index]  $\leftarrow$  array[ $i_1$ ];  
             $i_1 \leftarrow i_1 + 1$ ;  
        else  
            copyBuffer[index]  $\leftarrow$  array[ $i_2$ ];  
             $i_2 \leftarrow i_2 + 1$ ;  
        end  
    end  
    for  $index \leftarrow low$  to  $high$  do  
        array[index]  $\leftarrow$  copyBuffer[index];  
    end  
end  
Function MergeHelper( $low, high$ )  
    if  $low < high$  then  
        middle  $\leftarrow \frac{(low+high)}{2}$ ;  
        MergeHelper( $low, middle$ );  
        MergeHelper( $middle + 1, high$ );  
        Merge( $low, middle, high$ );  
    end  
end  
MergeHelper( $0, n-1$ );
```

Algorithm 3: Merge Sort

Data: An array of non-negative numbers (0-indexed). n is the length of the array.

Result: A sorted array of numbers.

```
Function Rearrange(first, last)
    index  $\leftarrow$  first;
    pivot  $\leftarrow$  array[first];
    while first  $\leq$  last do
        if array[first]  $\leq$  pivot then
            first  $\leftarrow$  first + 1;
        else if array[last]  $\geq$  pivot then
            last  $\leftarrow$  last - 1;
        else
            swap(first, last);
            first  $\leftarrow$  first + 1;
            last  $\leftarrow$  last - 1;
        end
    end
    swap(index, last);
end

Function QuickHelper(first, last)
    if first < last then
        index  $\leftarrow$  Rearrange(first, last);
        QuickHelper(first, index - 1);
        QuickHelper(index - 1, last);
    end
end

QuickHelper(0,  $n - 1$ );
```

Algorithm 4: Quick Sort

Using these algorithms, we performed various empirical tests. The tests are listed below as follows:

1. All sorts on $n \in \{500, 1000, 2000, 4000, 8000\}$ with random data.
2. Each sort with $n \in \{500, 1000, 2000, 4000, 8000\}$ for random, ascending, descending.
3. Each sort on random data for $n \in \{100i : i \in \mathbb{Z}, i \in [1, 100]\}$. In other words, $n \in \{100, 200, \dots, 10000\}$.
4. Each sort ran on random data with size $n = 500$ a total of 5000 times.

Modifications were made to the original Jar using the decompiled form. All sorting implementations remain untouched throughout the experiment.

2.2 Results

We will now demonstrate our empirical results. The following graphs were generated using Google Sheets.

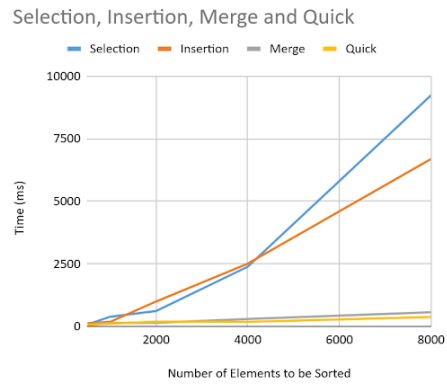


Figure 1: Comparison between all sorts on random data.

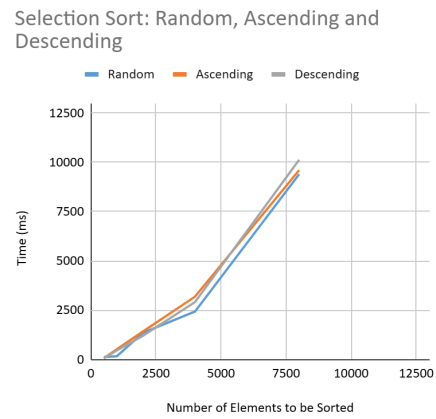


Figure 2: Selection sort on random, ascending, descending data.

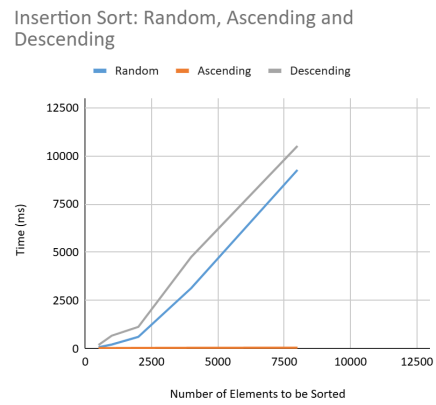


Figure 3: Insertion sort on random, ascending, descending data.

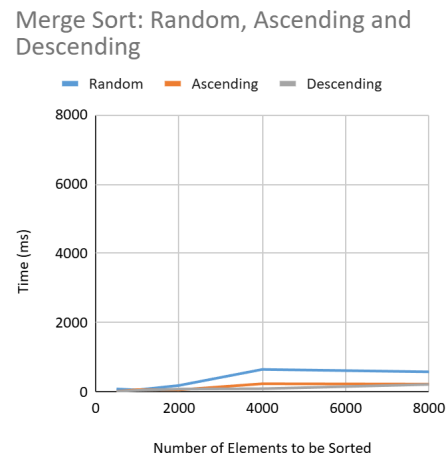


Figure 4: Merge sort on random, ascending, descending data.

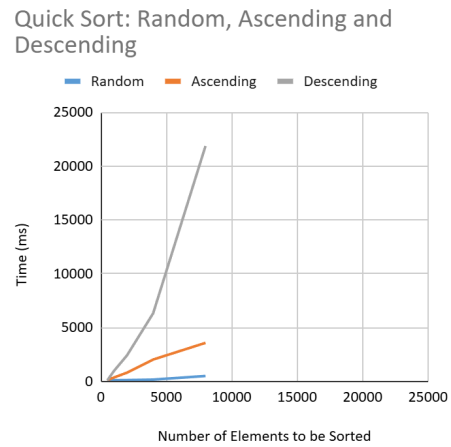


Figure 5: Quick sort on random, ascending, descending data.

We now consider more data points and time variations on the same data. These graphs were generated by Python Matplotlib.

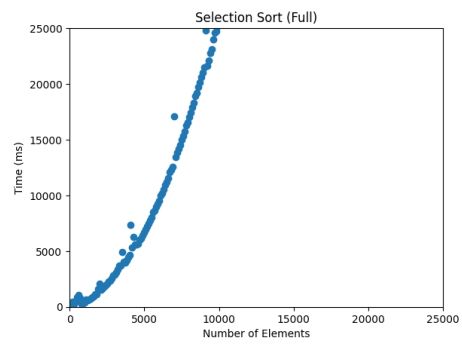


Figure 6: Selection sort random ordered data with more elements.

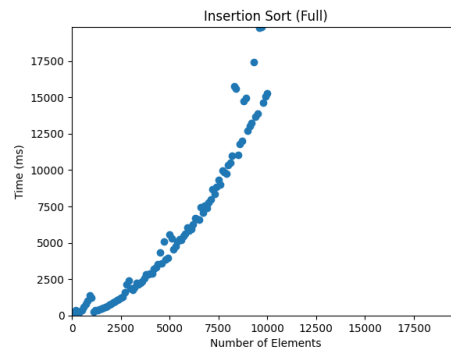


Figure 7: Insertion sort random ordered data with more elements.

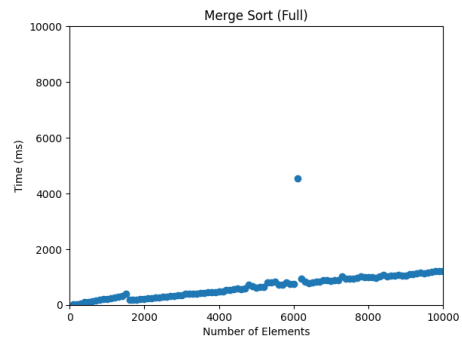


Figure 8: Merge sort random ordered data with more elements.

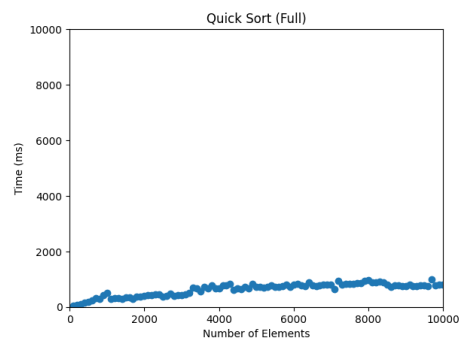


Figure 9: Quick sort random ordered data with more elements.

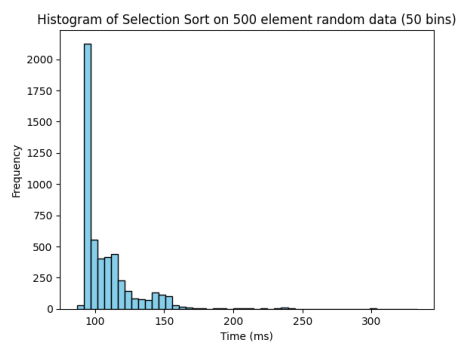


Figure 10: Selection sort 5000 times on $n = 500$.

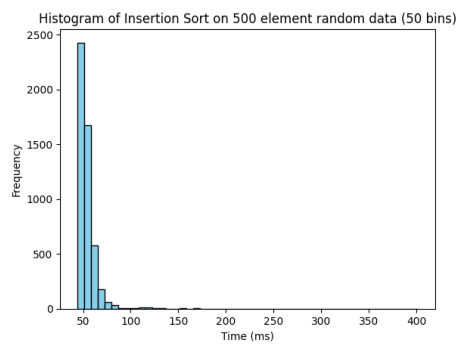


Figure 11: Insertion sort 5000 times on $n = 500$.

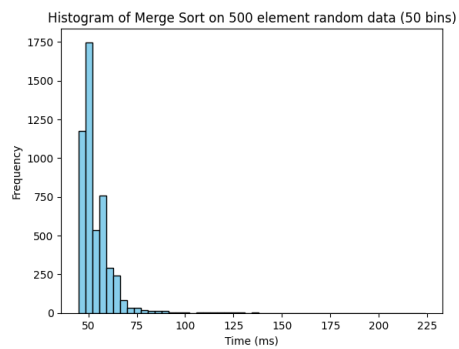


Figure 12: Merge sort 5000 times on $n = 500$.

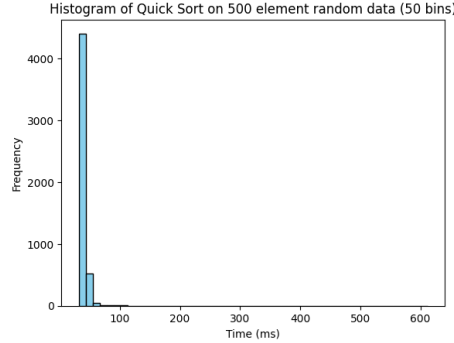


Figure 13: Quick sort 5000 times on $n = 500$.

2.3 Statistical Analysis

Upon a statistical analysis, we can properly analyze the approximate big-O of these sorting algorithms. We took into account the ratios as well as how statistical tests fit. We will denote the empirical big-O as $\mathcal{O}(f(n))$ rather than $O(f(n))$.

Definition 2.1 (Empirical Time Complexity). A relation $f(n)$ is $\mathcal{O}(g_1(n))$ on a family of functions $\{g_1(n), g_2(n), \dots\}$ if and only if $g_1(n)$ fits $f(n)$ with the highest R^2 value among the family and the confidence value p is statistically significant.

Remark. Note that our definition of $\mathcal{O}$ is more similar to Θ than O .

We will choose our family as $\{1, \log n, n \log n, n^2, n^3\}$ as these are common complexities on sorting algorithms. We also had combinations of different functions in the family, as well as of course coefficients.

2.3.1 Selection Sort

We report that selection sort is $\mathcal{O}(316 + 0.122n + 0.000246n^2) \approx \mathcal{O}(n^2)$. We have $R^2 = 0.9935$ and $p < 2.2 \times 10^{-16}$. The n^2 term was very significant given it had $p < 2 \times 10^{-16}$, further backing our claims. All other models had either lower R^2 and/or less significance.

2.3.2 Insertion Sort

We establish that insertion sort is $\mathcal{O}(511 - 0.134n + 0.000179n^2) \approx \mathcal{O}(n^2)$. We have $R^2 = 0.9594$ and $p < 2.2 \times 10^{-16}$. The n^2 term was very significant given it had $p < 2 \times 10^{-16}$, further backing our claims. All other models had either lower R^2 and/or less significance.

2.3.3 Merge Sort

We claim that merge sort is $\mathcal{O}(56.06 + 0.123n) \approx \mathcal{O}(n)$. While this result may seem counterintuitive, it had overall the highest significance on the data tested.

Remark. It is important to note that there may have been a computational error for this part. There is a possibility that the logarithmic regression used failed to work properly, so please interpret with caution.

2.3.4 Quick Sort

We claim that quick sort is $\mathcal{O}(96.46 + 0.184n + 0.0000116n^2) \approx \mathcal{O}(n^2)$. While this result may seem counterintuitive, it had overall the highest significance on the data tested. It may also be possible that the code was choosing bad pivots on average.

Remark. It is important to note that there may have been a computational error for this part. There is a possibility that the logarithmic regression used failed to work properly, so please interpret with caution.

3 Theoretical Comparison

We will perform a theoretical analysis on the four sorting algorithms as for a comparison to the empirical study.

3.1 Selection Sort

Theorem 3.1. *Selection sort is $\Theta(n^2)$ in all cases.*

Proof. Recall that selection sort consists of finding either the maximum or minimum and swapping it to the correct location. Let $T(n)$ denote the number of operations. Our design of $T(n)$ intends for selection sort to be $\Theta(T(n))$. We can see that

$$T(n) = \sum_{0 \leq i < n} \sum_{i \leq j < n} c$$

where n denotes the size of the array and c is a constant. This $T(n)$ representation is fairly trivial because looping through the array is taking indexes $[0, n)$ and finding the max for each is about $[i, n)$. Since selection sort is fairly stable, the only variations in $T(n)$ between cases is with the constants. The proof of that is fairly simple, finding min/max and looping through is always the same. Now, we can rewrite $T(n)$ as

$$T(n) = \frac{c(n+1)(n)}{2}$$

through basic summation rules. Now it is clear that $\Theta(T(n)) = \Theta(n^2)$. $\square$

3.2 Insertion Sort

Lemma 3.2. *The worst case of insertion sort is $\Theta(n^2)$*

Proof. When the array is in reversed order, this proof is very close to selection sort. However, instead of finding max/min, one needs to swap the current element down to the front/end. Therefore approximately the same $T(n)$ function of selection sort applies. $\square$

Lemma 3.3. *The best case of insertion sort is $\Theta(n)$.*

Proof. This case entails that the array is in order already. In this case, no swaps are required, so it simply iterates through giving us $\Theta(n)$. $\square$

Theorem 3.4. *The average case of insertion sort is $\Theta(n^2)$. [08]*

Proof. We must first consider *inversions*. An inversion occurs in a permutation when two elements are out of their natural order. More formally, let π be a permutation and $i < j$. An inversion occurs when $\pi(i) > \pi(j)$. We claim that the number of inversions is equivalent to the number of swaps. Suppose we must swap $A[i]$ and $A[i + 1]$. This means $A[i] > A[i + 1]$, but $i < i + 1$ obviously. Therefore, the two were an inversion. No other inversions must've been changed, which can easily be verified by considering the possibilities of the locations of the inversions. At the end of the sort, zero swaps are required and there are zero inversions, so therefore there are the same amount of inversions as swaps. And so the time complexity is equal to $\Theta(n + I)$ where I denotes the number of inversions. Let $X_{i,j} = 1$ if $A[i]$ and $A[j]$ form an inversion and 0 otherwise. We will need to consider $\frac{n(n-1)}{2}$ variables of the form $X_{i,j}$. The sum of all inversions I can be calculated as follows:

$$I = \sum_{0 \leq i < j < n} X_{i,j}$$

Since this is the average case, we need to determine the expected value of I , denoted as $\mathbb{E}[I]$. We can see that

$$\mathbb{E}[I] = \mathbb{E}[\sum X_{i,j}] = \sum \mathbb{E}[X_{i,j}]$$

by the linearity of expectation. We can see that $\mathbb{E}[X_{i,j}]$ is equivalent to the probability that $X_{i,j} = 1$, or in other words the probability that $A[i] > A[j]$. Ignoring the possibility of duplicates (which wouldn't affect the outcome too much), the probability is exactly $\frac{1}{2}$. Therefore we simply need to evaluate

$$\sum_{0 \leq i < j < n} \frac{1}{2} = \frac{n(n-1)}{4}$$

It follows that the average case is exactly $\Theta(n^2)$. $\square$

Corollary 3.4.1. *Insertion sort is $O(n^2)$ and $\Omega(n)$.*

Proof. Derived from the above lemmas. Note that $O(n^2)$ is used as the worst case context and $\Omega(n)$ is used for best case. $\square$

3.3 Merge Sort

Theorem 3.5. *Merge sort is $\Theta(n \log n)$ in all cases.*

Proof. We will apply the *Master Theorem* [UVA]. Let $T(n) = 2T(\frac{n}{2}) + f(n)$. Each section of the array is split into 2 sections with half of the length. Zipping each section is $\Theta(n)$, no matter which case. Let $\delta = \log_2(2) = 1$. Since $f(n)$ is $\Theta(n^\delta)$, $T(n)$ or merge sort is $\Theta(n \log n)$ as desired. $\square$

3.4 Quick Sort

We will let $T(n) = T(f(n)) + T(g(n)) + O(n)$ be the representation of quick sort where f and g depend on the placement of the pivot. Note that the physical swap process is $O(n)$.

Corollary 3.5.1. *Quick sort's average and best case is $\Theta(n \log n)$. [Qui25]*

Proof. In this case, the pivot works well so therefore the data is split into two halves, each of size $\frac{n}{2}$. So $f(n) = g(n) = \frac{n}{2}$. Thus this is the same $T(n)$ as merge sort, so the Master Theorem applies analogously. $\square$

Lemma 3.6. *The worst case complexity is exactly $\Theta(n^2)$ [Qui25]*

Proof. We can see that upon choosing a bad pivot, the data may divide as so

$$T(n) = T(0) + T(n-1) + O(n)$$

where $T(0)$ is a base case. This is clearly $\Theta(n^2)$. $\square$

4 Conclusion

We conclude that the empirical analysis on these sorting algorithms is varied and unlike the theoretical analysis. This fact may change with a better implementation and more tests in future studies. As the consistent $n = 500$ tests show, there is wildly different behavior on the same n value for all tests. Upon testing more consistently and more often, the empirical study may more closely align with the theoretical.

5 Acknowledgments

We would like to acknowledge our Computer Science teacher Nicole Kim for providing us with the needed materials and guiding us through this project.

References

- [08] *Stack overflow*. 2008. URL: <https://stackoverflow.com/>.
- [Qui25] Quick Sort. *Quick Sort — Wikipedia, The Free Encyclopedia*. 2025.

- [Tea] TJ CS Teachers. *EmpiricalBigO*.
[UVA] UVA. *CS4102 Algorithms*.